

Curso de CrewAI - Temario Completo

Table of Contents

1. Módulo 1: Introducción a CrewAI	1
1.1. 1.1 ¿Qué es CrewAI?	1
1.1.1. Definición Formal	1
1.1.2. Características Principales	1
1.1.3. Diferencias con Otros Frameworks	1
1.1.4. Casos de Uso Principales	1
1.1.5. Ventajas	2
1.1.6. Limitaciones	2
1.2. 1.2 Instalación y Configuración	2
1.2.1. Requisitos Previos	2
1.2.2. Instalación Paso a Paso	2
1.2.3. Configuración de API Keys	3
1.2.4. Estructura de Proyecto Recomendada	4
2. Módulo 2: Conceptos Fundamentales	5
2.1. 2.1 Agentes	5
2.1.1. Estructura Básica de un Agente	5
2.1.2. Componentes Clave	6
2.2. 2.2 Tareas	6
2.2.1. Estructura Básica de una Tarea	6
2.2.2. Parámetros Importantes	7
2.3. 2.3 Crews	7
2.3.1. Estructura Básica de un Crew	7
2.3.2. Procesos de Coordinación	8
3. Módulo 3: Herramientas (Tools)	9
3.1. 3.1 Herramientas Integradas	9
3.1.1. Herramientas Disponibles	9
3.2. 3.2 Herramientas Personalizadas	9
3.2.1. Opción 1: Decorador @tool	10
3.2.2. Opción 2: Clase BaseTool	10
3.3. 3.3 Integración de Herramientas	10
3.3.1. Registrar en Agentes	10
3.3.2. Tool Description	11
4. Módulo 4: Configuración Avanzada	12
4.1. 4.1 Configuración mediante YAML	12
4.2. 4.2 Memoria y Contexto	13
5. Módulo 5: Casos de Uso Prácticos	14
5.1. 5.1 Investigación y Análisis de Mercado	14
5.1.1. Arquitectura del Caso de Uso	14

5.1.2. Implementación	14
5.1.3. Casos de Uso Específicos	15
5.2. 5.2 Desarrollo de Software	15
5.2.1. Arquitectura	15
5.2.2. Implementación	16
5.3. 5.3 Marketing y Contenido	17
5.3.1. Arquitectura	17
5.3.2. Ejemplo de Implementación	17
5.4. 5.4 Análisis de Datos	18
5.4.1. Arquitectura	18
5.4.2. Implementación	19
6. Módulo 6: Monitoreo y Debugging	21
6.1. 6.1 Logging y Observabilidad	21
6.1.1. Logging Básico	21
6.1.2. Logger Personalizado	21
6.2. 6.2 Debugging de Ejecución	22
6.2.1. Debugging con Callbacks	23
6.3. 6.3 Evaluación de Resultados	23
7. Módulo 7: Mejores Prácticas	25
7.1. 7.1 Diseño de Agentes	25
7.1.1. Roles Específicos y Sin Ambigüedad	25
7.1.2. Backstories que Añadan Contexto	25
7.2. 7.2 Estructuración de Tareas	25
7.2.1. Dependencias Lógicas	26
7.3. 7.3 Optimización de Costos	26
7.3.1. Usar Modelos Más Pequeños	26
7.3.2. Limitar Iteraciones	27
7.3.3. Implementar Caching	27
8. Módulo 8: Escalabilidad y Arquitectura	28
8.1. 8.1 Crews Complejos	28
8.2. 8.2 Integración Web con FastAPI	28
9. Módulo 9: Troubleshooting	30
9.1. 9.1 Problemas Comunes	30
9.2. 9.2 Debugging Avanzado	31
10. Módulo 10: Proyecto Final	32
10.1. 10.1 Proyecto: Sistema de Análisis de Tendencias Multi-Fuente	32
10.1.1. Requisitos del Sistema	32
10.1.2. Arquitectura del Proyecto	32
10.1.3. Entregables Esperados	34
10.2. 10.2 Presentación y Validación	34
11. Anexo A: Comparativa de Frameworks	35

12. Anexo B: Recursos	36
13. Anexo C: Glosario	37
14. Anexos	38
14.1. A. Comparativa de Frameworks	38
14.2. B. Recursos	38
14.3. C. Glosario	38
14.4. D. Checklist de Proyecto	38

Chapter 1. Módulo 1: Introducción a CrewAI

1.1. 1.1 ¿Qué es CrewAI?

Concepto: CrewAI es un framework de código abierto que proporciona una abstracción de alto nivel para crear sistemas multi-agente coordinados. A diferencia de AutoGen y LangChain, CrewAI se enfoca específicamente en la orquestación de "equipos" de agentes (crews) que trabajan juntos de forma coordinada.

1.1.1. Definición Formal

CrewAI es un framework Python que permite:

- Definir agentes con roles, goals y backstories específicos
- Crear tareas con descripciones claras y outputs esperados
- Coordinar múltiples agentes en un "crew" con procesos definidos
- Integrar herramientas externas de forma declarativa
- Ejecutar flujos de trabajo complejos de forma resiliente

1.1.2. Características Principales

- **Agentes Especializados:** Cada agente tiene un rol definido, objetivo y contexto (backstory)
- **Tareas Declarativas:** Las tareas describen qué se debe hacer, no cómo hacerlo
- **Procesos de Coordinación:** Diferentes formas de orquestar agentes (secuencial, jerárquico, etc.)
- **Herramientas Integradas:** Búsqueda web, lectura de archivos, ejecución de código, etc.
- **Memoria Persistente:** Agentes pueden aprender de interacciones previas
- **Callbacks y Eventos:** Hooks para monitorizar y personalizar el comportamiento

1.1.3. Diferencias con Otros Frameworks

Aspecto	CrewAI	AutoGen	LangChain
Enfoque Principal	Orquestación de equipos	Conversaciones multi-agente	Cadenas de LLM
Declaratividad	Alto (YAML/Python)	Programático	Programático
Herramientas	Integradas	Manuales	Integradas
Memoria	Nativa	Manual	Limited
Curva de aprendizaje	Baja	Media	Media

1.1.4. Casos de Uso Principales

- **Investigación y Análisis:** Market research, análisis competitivo, recopilación de información
- **Desarrollo de Software:** Code review automático, análisis de arquitectura, testing
- **Marketing y Contenido:** Estrategia de contenido, generación de posts, adaptación multi-canal
- **Análisis de Datos:** Procesamiento de datasets, generación de reportes, visualización

- **Automatización Empresarial:** Procesos repetitivos, flujos de aprobación, documentación

1.1.5. Ventajas

- Sintaxis simple y intuitiva
- Integración con modelos locales (Ollama) y APIs (OpenAI, etc.)
- Sistema de herramientas flexible
- Memoria nativa para agentes
- Comunidad activa en GitHub

1.1.6. Limitaciones

- Menos maduro que LangChain
- Documentación en evolución
- Limitado a arquitecturas lineales/jerárquicas
- Requiere configuración explícita de herramientas

1.2. 1.2 Instalación y Configuración

1.2.1. Requisitos Previos

```
# Sistema operativo
# Linux, macOS, o Windows

# Verificar Python
python --version # 3.8 o superior

# Verificar pip
pip --version
```

1.2.2. Instalación Paso a Paso

Opción 1: Instalación Básica

```
# 1. Crear entorno virtual
python -m venv crewai_env

# 2. Activar entorno
# Linux/macOS
source crewai_env/bin/activate

# Windows
crewai_env\Scripts\activate
```

```
# 3. Instalar CrewAI
pip install crewai

# 4. Verificar instalación
python -c "import crewai; print(crewai.__version__)"
```

Opción 2: Instalación con Dependencias Extra

```
# Instalar con todas las herramientas
pip install crewai[tools]

# Incluye:
# - crewai-tools (herramientas integradas)
# - requests (para llamadas HTTP)
# - langchain (dependencia base)
```

Opción 3: Instalación desde Fuente

```
# Clonar repositorio
git clone https://github.com/joaomdmoura/crewai.git
cd crewai

# Instalar en modo desarrollo
pip install -e .
```

1.2.3. Configuración de API Keys

Para OpenAI:

```
# Opción 1: Variable de entorno
export OPENAI_API_KEY="sk-..."

# Opción 2: Archivo .env
echo "OPENAI_API_KEY=sk-..." > .env

# En Python
from dotenv import load_dotenv
load_dotenv()
```

Para Ollama Local (Recomendado):

```
# 1. Instalar Ollama
# https://ollama.ai

# 2. Ejecutar servidor
ollama serve
```

```
# 3. Descargar modelo
ollama pull mistral

# 4. En Python
from crewai import Agent
agent = Agent(
    role="Experto",
    goal="Ayudar",
    backstory="...",
    llm="ollama:mistral"
)
```

1.2.4. Estructura de Proyecto Recomendada

```
proyecto-crewai/
├── .env
├── .gitignore
├── requirements.txt
├── main.py
├── config/
│   ├── agents.yaml
│   └── tasks.yaml
├── agents/
│   ├── __init__.py
│   └── custom_agents.py
├── tools/
│   ├── __init__.py
│   └── custom_tools.py
├── crews/
│   ├── __init__.py
│   └── main_crew.py
└── output/
    └── (resultados)
```


Chapter 2. Módulo 2: Conceptos Fundamentales

2.1. 2.1 Agentes

Concepto: Un agente es la unidad fundamental en CrewAI. Representa un "miembro del equipo" con un rol específico, objetivo claro y contexto (backstory). Cada agente puede usar herramientas para lograr sus objetivos.

2.1.1. Estructura Básica de un Agente

```
from crewai import Agent

# Opción 1: Constructor básico
agent = Agent(
    role="Analista de Datos",
    goal="Analizar datasets y generar insights",
    backstory="Soy un experto en análisis de datos con 10 años de experiencia",
    verbose=True, # Mostrar reasoning
    allow_delegation=False, # No delega tareas
    tools=[tool1, tool2] # Herramientas disponibles
)

# Opción 2: Con configuración personalizada
agent = Agent(
    role="Investigador",
    goal="Recopilar información relevante",
    backstory="Soy un investigador meticuloso",
    llm="gpt-4", # Modelo específico
    max_iter=5, # Máximo de iteraciones
    memory=True, # Habilitar memoria
    tools=tools_list
)

# Opción 3: Herencia para especialización
class DataAnalystAgent(Agent):
    def __init__(self):
        super().__init__(
            role="Analista de Datos Senior",
            goal="Generar reportes analíticos detallados",
            backstory="Especialista en análisis cuantitativo",
            tools=[analysis_tool, visualization_tool]
        )
```

2.1.2. Componentes Clave

1. Role: Descripción del rol del agente

```
role = "Experto en Python" # Claro y específico
```

2. Goal: Objetivo que el agente intenta lograr

```
goal = "Escribir código Python limpio y eficiente que resuelva el problema"
```

3. Backstory: Contexto y experiencia (afecta razonamiento)

```
backstory = "Soy un desarrollador Python con 15 años de experiencia en sistemas distribuidos"
```

4. Tools: Herramientas disponibles para el agente

```
tools = [search_tool, code_executor, file_reader]
```

2.2. 2.2 Tareas

Concepto: Una tarea es una unidad de trabajo específica asignada a un agente. Describe QUÉ debe hacer, no CÓMO hacerlo.

2.2.1. Estructura Básica de una Tarea

```
from crewai import Task

# Opción 1: Constructor básico
task = Task(
    description="Analiza el dataset customer.csv e identifica tendencias principales",
    expected_output="Reporte con 5 tendencias principales y recomendaciones",
    agent=data_analyst_agent
)

# Opción 2: Con dependencias
task2 = Task(
    description="Basado en el análisis anterior, genera un plan de acción",
    expected_output="Plan de acción detallado con 10 pasos",
    agent=strategist_agent,
    depends_on=[task1] # Espera a que task1 termine
)

# Opción 3: Con validación de output
task3 = Task(
```

```

description="Revisa el código y sugiere mejoras",
expected_output="Reporte de code review con 5-10 sugerencias",
agent=code_reviewer_agent,
validation="output.json", # Archivo de validación
callback=log_callback
)

```

2.2.2. Parámetros Importantes

- **description:** Descripción clara de la tarea
- **expected_output:** Qué se espera obtener (formato, estructura)
- **agent:** Agente responsable
- **depends_on:** Tareas que debe esperar
- **callback:** Función a ejecutar cuando termine

2.3. 2.3 Crews

Concepto: Un crew es el "equipo" de agentes que trabajan coordinadamente. Define cómo interactúan los agentes y en qué orden ejecutan sus tareas.

2.3.1. Estructura Básica de un Crew

```

from crewai import Crew, Process

# Opción 1: Crew secuencial simple
crew = Crew(
    agents=[agent1, agent2, agent3],
    tasks=[task1, task2, task3],
    process=Process.sequential # Ejecuta tareas una tras otra
)

# Opción 2: Crew jerárquico
crew = Crew(
    agents=[researcher, analyst, manager],
    tasks=[research_task, analysis_task, summary_task],
    process=Process.hierarchical,
    manager_agent=project_manager # Agente que coordina
)

# Opción 3: Crew con callbacks
def task_callback(output):
    print(f"Tarea completada: {output.description}")

crew = Crew(
    agents=agents,
    tasks=tasks,
    process=Process.sequential,

```

```
        verbose=True,  
        callback=task_callback  
    )  
  
    # Ejecutar crew  
    result = crew.kickoff(inputs={"query": "Analiza el mercado de IA"})
```

2.3.2. Procesos de Coordinación

Sequential: Tareas se ejecutan una después de otra

```
process=Process.sequential
```

Hierarchical: Manager agent coordina y delega

```
process=Process.hierarchical  
manager_agent=ceo_agent
```

Custom: Definir orden personalizado

```
# Las dependencias en tasks definen el orden
```

Chapter 3. Módulo 3: Herramientas (Tools)

3.1. 3.1 Herramientas Integradas

Concepto: CrewAI proporciona herramientas integradas que los agentes pueden usar para realizar acciones específicas. Son funcionalidades preestructuradas que se pueden incorporar directamente.

3.1.1. Herramientas Disponibles

1. Búsqueda Web (SerperDevTool)

```
from crewai_tools import SerperDevTool

search_tool = SerperDevTool()
# Requiere API key de Serper
```

2. Lectura de Archivos (FileReadTool)

```
from crewai_tools import FileReadTool

file_reader = FileReadTool(
    file_path="datos.csv"
)
```

3. Escritura de Archivos (FileWriteTool)

```
from crewai_tools import FileWriteTool

file_writer = FileWriteTool(
    file_path="output.txt"
)
```

4. Ejecución de Código (CodeInterpreterTool)

```
from crewai_tools import CodeInterpreterTool

code_tool = CodeInterpreterTool(
    sandbox_type="local" # 0 "docker"
)
```

3.2. 3.2 Herramientas Personalizadas

Concepto: Crear tus propias herramientas para funcionalidades específicas del dominio.

3.2.1. Opción 1: Decorador @tool

```
from crewai_tools import tool

@tool
def calculate_average(numbers: list) -> float:
    \"\"\"Calcula el promedio de una lista de números\"\"\"
    if not numbers:
        return 0
    return sum(numbers) / len(numbers)

@tool
def fetch_user_data(user_id: int) -> dict:
    \"\"\"Obtiene datos del usuario de la base de datos\"\"\"
    # Implementación
    return {"id": user_id, "name": "Usuario"}
```

3.2.2. Opción 2: Clase BaseTool

```
from crewai_tools import BaseTool

class DataProcessingTool(BaseTool):
    name: str = "data_processor"
    description: str = "Procesa datos con transformaciones específicas"

    def _run(self, data: str) -> str:
        # Implementación
        return f"Procesado: {data}"

# Usar
tool = DataProcessingTool()
```

3.3. Integración de Herramientas

3.3.1. Registrar en Agentes

```
agent = Agent(
    role="Analista",
    goal="Analizar datos",
    backstory="...",
    tools=[search_tool, file_reader, calculate_average]
)
```

3.3.2. Tool Description

Las descripciones son críticas para que el agente sepa cuándo usarlas:

```
@tool("calculate_stats")
def calculate_statistics(data: list) -> dict:
    \"\"\"
    Calcula estadísticas descriptivas de un dataset.
    Útil para: análisis exploratorio, resumen de datos numéricos.
    Retorna: media, mediana, desviación estándar.
    \"\"\"
    return {"mean": 0, "median": 0, "std": 0}
```

Chapter 4. Módulo 4: Configuración Avanzada

4.1. 4.1 Configuración mediante YAML

Concepto: Separar la configuración de agentes y tareas en archivos YAML para mayor mantenibilidad.

agents.yaml:

```
researcher:
  role: "Investigador Senior"
  goal: "Recopilar información de alta calidad sobre tópicos específicos"
  backstory: |
    Eres un investigador experimentado con acceso a múltiples fuentes.
    Tu capacidad para encontrar información relevante es excepcional.

analyst:
  role: "Analista de Datos"
  goal: "Analizar información y generar insights accionables"
  backstory: |
    Eres un analista con experiencia en transformar datos en decisiones.
```

tasks.yaml:

```
research_task:
  description: "Investiga el mercado de IA en 2024"
  expected_output: "Reporte comprensivo con 10 tendencias principales"
  agent: "researcher"

analysis_task:
  description: "Analiza los hallazgos de la investigación"
  expected_output: "Análisis con recomendaciones estratégicas"
  agent: "analyst"
  depends_on:
    - research_task
```

Carga en Python:

```
from crewai import Crew
from crewai.configs import AgentConfig, TaskConfig

crew = Crew.from_configs(
    agents_config=AgentConfig.load_from_yaml("agents.yaml"),
    tasks_config=TaskConfig.load_from_yaml("tasks.yaml")
```


)

4.2. 4.2 Memoria y Contexto

Memoria Nativa de CrewAI:

```
# Habilitar memoria
agent = Agent(
    role="Asistente",
    goal="Ayudar al usuario",
    backstory="...",
    memory=True, # Habilita memoria
    long_term_memory=True # Memoria persistente
)

# Acceder a memoria
agent.memory.get_memory()
agent.memory.save_memory(key="context", value="información")
```

Chapter 5. Módulo 5: Casos de Uso Prácticos

5.1. 5.1 Investigación y Análisis de Mercado

Objetivo: Crear un sistema automático para investigar mercados, analizar tendencias y generar reportes ejecutivos.

5.1.1. Arquitectura del Caso de Uso

```
Entrada (Query)
├─ [Investigador] → Búsqueda de tendencias
├─ [Analista] → Análisis de datos
├─ [Reportero] → Generación de reporte
└─ Salida (Reporte Ejecutivo)
```

5.1.2. Implementación

```
from crewai import Agent, Task, Crew, Process

# 1. Crear agentes especializados
researcher = Agent(
    role="Investigador de Mercado",
    goal="Identificar tendencias emergentes y oportunidades",
    backstory="Especialista con 10 años en investigación de mercados",
    tools=[search_tool]
)

analyst = Agent(
    role="Analista Estratégico",
    goal="Generar insights accionables de la información",
    backstory="Analista con experiencia en datos empresariales"
)

reporter = Agent(
    role="Redactor Ejecutivo",
    goal="Presentar hallazgos en formato profesional",
    backstory="Comunicador especializado en reportes ejecutivos"
)

# 2. Crear tareas coordinadas
research_task = Task(
    description="Investiga las tendencias del mercado de {topic}",
    expected_output="Reporte con 5-10 tendencias principales",
```

```

        agent=researcher
    )

    analysis_task = Task(
        description="Analiza las tendencias y genera recomendaciones",
        expected_output="Análisis con implicaciones estratégicas",
        agent=analyst,
        depends_on=[research_task]
    )

    report_task = Task(
        description="Genera un reporte ejecutivo con conclusiones",
        expected_output="Reporte de 1-2 páginas con conclusiones",
        agent=reporter,
        depends_on=[analysis_task]
    )

    # 3. Crear crew y ejecutar
    crew = Crew(
        agents=[researcher, analyst, reporter],
        tasks=[research_task, analysis_task, report_task],
        process=Process.sequential
    )

    result = crew.kickoff(inputs={"topic": "Inteligencia Artificial 2024"})
    print(result.raw_output)

```

5.1.3. Casos de Uso Específicos

- **Market Research:** Análisis de nuevos mercados
- **Competitive Analysis:** Monitoreo de competencia
- **Trend Forecasting:** Predicción de tendencias
- **Industry Reports:** Reportes por industria

5.2. 5.2 Desarrollo de Software

Objetivo: Crear un sistema para revisar código, analizar arquitectura y sugerir mejoras.

5.2.1. Arquitectura

Código fuente

□

[Code Reviewer] → Análisis de calidad

□

[Architect] → Análisis de diseño

□

[Security Auditor] → Verificación de seguridad

5.2.2. Implementación

```
code_reviewer = Agent(
    role="Code Reviewer Senior",
    goal="Revisar código y sugerir mejoras de calidad",
    backstory="Desarrollador con 15 años en code review",
    tools=[file_reader_tool]
)

architect = Agent(
    role="Arquitecto de Software",
    goal="Analizar arquitectura y proponer mejoras",
    backstory="Experto en patrones de diseño"
)

security_auditor = Agent(
    role="Security Auditor",
    goal="Identificar vulnerabilidades de seguridad",
    backstory="Especialista en seguridad de aplicaciones"
)

# Tareas
review_task = Task(
    description="Revisa el código y sugiere mejoras",
    expected_output="Reporte con 10-15 sugerencias",
    agent=code_reviewer
)

architecture_task = Task(
    description="Analiza la arquitectura general",
    expected_output="Recomendaciones de mejora arquitectónica",
    agent=architect,
    depends_on=[review_task]
)

security_task = Task(
    description="Audita seguridad del código",
    expected_output="Reporte de vulnerabilidades y fixes",
    agent=security_auditor
)

crew = Crew(
    agents=[code_reviewer, architect, security_auditor],
    tasks=[review_task, architecture_task, security_task],
    process=Process.sequential
)
```

5.3. 5.3 Marketing y Contenido

Objetivo: Crear una estrategia de contenido multi-canal coordinada.

5.3.1. Arquitectura

```
Estrategia de contenido
├─ [Content Strategist] → Plan de contenido
├─ [Content Writer] → Generación de posts
├─ [Social Media Manager] → Adaptación de canales
├─ [SEO Specialist] → Optimización
└─ Contenido multi-canal listo
```

5.3.2. Ejemplo de Implementación

```
strategist = Agent(
    role="Content Strategist",
    goal="Definir estrategia de contenido integral",
    backstory="Especialista en marketing digital"
)

writer = Agent(
    role="Content Writer",
    goal="Crear contenido de alta calidad",
    backstory="Escritor especializado en blogs técnicos"
)

social_manager = Agent(
    role="Social Media Manager",
    goal="Adaptar contenido para redes sociales",
    backstory="Community manager con 8 años de experiencia"
)

seo_specialist = Agent(
    role="SEO Specialist",
    goal="Optimizar contenido para buscadores",
    backstory="Experto en SEO y keywords"
)

# Tareas coordinadas
strategy_task = Task(
    description="Crea una estrategia de contenido para {topic}",
    expected_output="Plan de contenido con 10 posts",
```

```

        agent=strategist
    )

    writing_task = Task(
        description="Escribe posts según la estrategia",
        expected_output="5 posts de blog completos",
        agent=writer,
        depends_on=[strategy_task]
    )

    social_task = Task(
        description="Adapta los posts para redes sociales",
        expected_output="Versiones de posts para Twitter, LinkedIn, Instagram",
        agent=social_manager,
        depends_on=[writing_task]
    )

    seo_task = Task(
        description="Optimiza todo el contenido para SEO",
        expected_output="Contenido optimizado con keywords",
        agent=seo_specialist,
        depends_on=[social_task]
    )

    crew = Crew(
        agents=[strategist, writer, social_manager, seo_specialist],
        tasks=[strategy_task, writing_task, social_task, seo_task],
        process=Process.sequential
    )

```

5.4. 5.4 Análisis de Datos

Objetivo: Pipeline automático para exploración, análisis y visualización de datasets.

5.4.1. Arquitectura

```

Dataset raw
├──
├── [Data Cleaner] → Limpieza y preparación
├──
├── [Data Analyst] → Análisis exploratorio
├──
├── [Statistician] → Análisis estadístico
├──
├── [Data Visualizer] → Reportes visuales
├──
└── Insights y recomendaciones

```

5.4.2. Implementación

```
data_cleaner = Agent(
    role="Data Cleaner",
    goal="Limpiar y preparar datos",
    backstory="Especialista en data preparation"
)

analyst = Agent(
    role="Data Analyst",
    goal="Realizar análisis exploratorio",
    backstory="Analista con experiencia en datasets grandes"
)

statistician = Agent(
    role="Statistician",
    goal="Realizar análisis estadístico riguroso",
    backstory="PhD en Estadística aplicada"
)

visualizer = Agent(
    role="Data Visualization Expert",
    goal="Crear visualizaciones claras e impactantes",
    backstory="Diseñador especializado en data viz"
)

# Tareas del pipeline
cleaning_task = Task(
    description="Limpia y prepara el dataset {filename}",
    expected_output="Dataset limpio sin valores faltantes",
    agent=data_cleaner
)

analysis_task = Task(
    description="Realiza análisis exploratorio",
    expected_output="Distribuciones, correlaciones, outliers",
    agent=analyst,
    depends_on=[cleaning_task]
)

statistical_task = Task(
    description="Realiza análisis estadístico",
    expected_output="Pruebas estadísticas y conclusiones",
    agent=statistician,
    depends_on=[analysis_task]
)

visualization_task = Task(
    description="Crea visualizaciones de los análisis",
    expected_output="Gráficos y dashboard",
```

```
        agent=visualizer,  
        depends_on=[statistical_task]  
    )  
  
    crew = Crew(  
        agents=[data_cleaner, analyst, statistician, visualizer],  
        tasks=[cleaning_task, analysis_task, statistical_task, visualization_task],  
        process=Process.sequential  
    )
```


Chapter 6. Módulo 6: Monitoreo y Debugging

6.1. 6.1 Logging y Observabilidad

Concepto: CrewAI proporciona herramientas para monitorizar la ejecución de crews y obtener visibilidad sobre qué está haciendo cada agente.

6.1.1. Logging Básico

```
from crewai import Crew

crew = Crew(
    agents=agents,
    tasks=tasks,
    verbose=True, # Mostrar todos los pasos en consola
    output_log_file="crew_execution.log" # Guardar logs en archivo
)

result = crew.kickoff()
```

6.1.2. Logger Personalizado

```
import logging
from datetime import datetime

class CrewLogger:
    def __init__(self, verbose=True):
        self.verbose = verbose
        self.logs = []
        self.execution_trace = []

    def log_task_start(self, task_name, agent_name):
        log_entry = {
            "timestamp": datetime.now().isoformat(),
            "event": "task_start",
            "task": task_name,
            "agent": agent_name
        }
        self.logs.append(log_entry)
        if self.verbose:
            print(f"[TASK START] {task_name} - {agent_name}")

    def log_task_end(self, task_name, status):
        log_entry = {
            "timestamp": datetime.now().isoformat(),
            "event": "task_end",
            "task": task_name,
```

```

        "status": status
    }
    self.logs.append(log_entry)
    if self.verbose:
        print(f"[TASK END] {task_name} - Status: {status}")

def log_error(self, error_message):
    log_entry = {
        "timestamp": datetime.now().isoformat(),
        "event": "error",
        "message": error_message
    }
    self.logs.append(log_entry)

def export_logs(self, filename):
    import json
    with open(filename, 'w') as f:
        json.dump(self.logs, f, indent=2)

# Uso
logger = CrewLogger(verbose=True)
logger.log_task_start("research", "Investigador")
logger.log_task_end("research", "completed")
logger.export_logs("crew_logs.json")

```

6.2. 6.2 Debugging de Ejecución

Obtener información detallada de ejecución:

```

# Ejecutar crew
result = crew.kickoff()

# Acceder a resultados
print("Output bruto:")
print(result.raw_output)

print("\nOutput de tareas:")
for task_output in result.tasks_output:
    print(f"Tarea: {task_output.description}")
    print(f"Output: {task_output.raw_output}")
    print(f"Agente: {task_output.agent}")

# Acceder a métricas
print(f"\nTiempo de ejecución: {result.execution_time}s")
print(f"Tokens usados: {result.token_count}")

```

6.2.1. Debugging con Callbacks

```
def on_task_start(task_name, agent_name):
    print(f"→ Iniciando tarea: {task_name}")

def on_task_complete(task_name, output):
    print(f"✓ Completada: {task_name}")
    print(f"  Output: {output[:100]}...")

def on_error(error):
    print(f"⚠ Error: {error}")

crew = Crew(
    agents=agents,
    tasks=tasks,
    on_task_start_callback=on_task_start,
    on_task_complete_callback=on_task_complete,
    on_error_callback=on_error
)
```

6.3. 6.3 Evaluación de Resultados

Validar la calidad de outputs:

```
def validate_output(task_output, expected_format=None):
    """Valida si el output cumple con criterios"""

    # Verificar que el output no esté vacío
    if not task_output or len(task_output.strip()) == 0:
        return False, "Output vacío"

    # Verificar longitud mínima
    if len(task_output.split()) < 10:
        return False, "Output muy corto"

    # Verificar formato si es especificado
    if expected_format == "json":
        try:
            import json
            json.loads(task_output)
        except:
            return False, "Formato JSON inválido"

    return True, "Output válido"

# Usar con crew
result = crew.kickoff()
for task_output in result.tasks_output:
```

```
valid, message = validate_output(task_output.raw_output)
print(f"{task_output.description}: {'✓' if valid else ''} {message}")
```

Chapter 7. Módulo 7: Mejores Prácticas

7.1. 7.1 Diseño de Agentes

Principios de diseño efectivo:

7.1.1. Roles Específicos y Sin Ambigüedad

```
# ✓ BIEN: Rol específico
role = "Analista de Tendencias de Mercado Junior"

# ❌ MAL: Rol demasiado genérico
role = "Analista"

# ✓ BIEN: Goal claro y medible
goal = "Identificar 5-10 tendencias emergentes en el mercado de IA"

# ❌ MAL: Goal vago
goal = "Analizar el mercado"
```

7.1.2. Backstories que Añadan Contexto

```
# ✓ BIEN: Backstory detallado y creíble
backstory = """
Eres un especialista en análisis de mercado con 10 años de experiencia
en tecnología. Has publicado 20+ reportes sobre tendencias de IA en
Forbes, TechCrunch y medios especializados. Tienes acceso a bases de
datos propietarias y contactos en empresas Fortune 500.
"""

# ❌ MAL: Backstory genérico
backstory = "Eres un analista"
```

7.2. 7.2 Estructuración de Tareas

Mejores prácticas para definir tareas:

```
# ✓ BIEN: Descripción detallada
task = Task(
    description="""
    Investiga las tendencias emergentes del mercado de IA en 2024.
    Busca información sobre: modelos de IA, aplicaciones nuevas,
    regulaciones, y empresas líderes. Incluye datos de al menos 3 fuentes.
    """,
    expected_output="""
```

Reporte en markdown con:

```
.
- 5-10 tendencias principales
- Análisis de impacto (alto/medio/bajo)
- Empresas afectadas
- Proyecciones para 2025
"""
agent=researcher
)

# MAL: Descripción vaga
task = Task(
    description="Investiga IA",
    expected_output="Reporte",
    agent=researcher
)
```

7.2.1. Dependencias Lógicas

```
# Orden correcto de dependencias
task1 = Task(
    description="Investigar datos",
    expected_output="Datos brutos",
    agent=researcher
)

task2 = Task(
    description="Analizar datos investigados",
    expected_output="Análisis",
    agent=analyst,
    depends_on=[task1] # Espera a investigación
)

task3 = Task(
    description="Escribir reporte basado en análisis",
    expected_output="Reporte ejecutivo",
    agent=writer,
    depends_on=[task2] # Espera a análisis
)
```

7.3. Optimización de Costos

Estrategias para reducir costos de API:

7.3.1. Usar Modelos Más Pequeños

```
# ✓ Para tareas simples: Modelos más pequeños y rápidos
```

```

simple_agent = Agent(
    role="Clasificador",
    goal="Clasificar documentos",
    backstory="...",
    llm="gpt-3.5-turbo" # Más económico que gpt-4
)

# ✓ Para tareas complejas: Modelos más poderosos
complex_agent = Agent(
    role="Analista Estratégico",
    goal="Generar estrategia competitiva",
    backstory="...",
    llm="gpt-4"
)

```

7.3.2. Limitar Iteraciones

```

agent = Agent(
    role="Investigador",
    goal="...",
    backstory="...",
    max_iterations=3 # Limitar intentos
)

```

7.3.3. Implementar Caching

```

import hashlib
import json

class TaskCache:
    def __init__(self):
        self.cache = {}

    def get_cache_key(self, task_desc, agent_role):
        combined = f"{task_desc}_{agent_role}"
        return hashlib.md5(combined.encode()).hexdigest()

    def get(self, task_desc, agent_role):
        key = self.get_cache_key(task_desc, agent_role)
        return self.cache.get(key)

    def set(self, task_desc, agent_role, result):
        key = self.get_cache_key(task_desc, agent_role)
        self.cache[key] = result

cache = TaskCache()

```

Chapter 8. Módulo 8: Escalabilidad y Arquitectura

8.1. 8.1 Crews Complejos

Arquitectura de múltiples crews coordinados:

```
# Crew 1: Investigación
research_crew = Crew(
    agents=[researcher],
    tasks=[research_task],
    process=Process.sequential
)

# Crew 2: Análisis
analysis_crew = Crew(
    agents=[analyst],
    tasks=[analysis_task],
    process=Process.sequential
)

# Crew 3: Reporting (usa outputs de otros crews)
reporting_crew = Crew(
    agents=[reporter],
    tasks=[reporting_task],
    process=Process.sequential
)

# Ejecutar secuencialmente
research_result = research_crew.kickoff()
analysis_result = analysis_crew.kickoff(inputs={"research": research_result})
final_result = reporting_crew.kickoff(inputs={"analysis": analysis_result})
```

8.2. 8.2 Integración Web con FastAPI

Exponer crews como API REST:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="CrewAI API")

class TaskInput(BaseModel):
    topic: str
    analysis_type: str
```



```

@app.post("/api/research")
async def research_endpoint(input_data: TaskInput):
    """Ejecutar crew de investigación"""
    crew = Crew(
        agents=[researcher, analyst],
        tasks=[research_task, analysis_task],
        process=Process.sequential
    )

    result = crew.kickoff(inputs={
        "topic": input_data.topic
    })

    return {
        "status": "completed",
        "result": result.raw_output,
        "metadata": {
            "tokens_used": result.token_count,
            "execution_time": result.execution_time
        }
    }

@app.get("/health")
async def health_check():
    return {"status": "healthy"}

# Ejecutar: uvicorn app:app --reload

```

Chapter 9. Módulo 9: Troubleshooting

9.1. 9.1 Problemas Comunes

Agente no selecciona la herramienta correcta:

```
# PROBLEMA: Herramientas mal descritas
@tool
def search_web(query):
    """Busca en la web""" # Descripción muy vaga
    pass

# SOLUCIÓN: Descripciones claras
@tool
def search_web(query: str) -> str:
    """
    Busca información en internet usando Serper API.
    Útil para: investigación de mercado, búsqueda de tendencias,
    obtener información actualizada.
    Retorna: resultados relevantes con URLs.
    """
    pass
```

Tareas que tardan demasiado:

```
# PROBLEMA
task = Task(
    description="Analiza todo el internet sobre IA",
    expected_output="Análisis completo",
    agent=analyst
)

# SOLUCIÓN: Ser específico y limitado
task = Task(
    description="""
    Analiza las 10 noticias más recientes sobre IA en los últimos 7 días.
    Enfócate en: regulación, tendencias tecnológicas, y adopción empresarial.
    """,
    expected_output="Resumen de 3-5 párrafos",
    agent=analyst
)
```

Outputs sin formato esperado:

```
# PROBLEMA
task = Task(
```

```

        description="Analiza datos",
        expected_output="Reporte"
    )

# SOLUCIÓN: Especificar exactamente el formato
task = Task(
    description="Analiza el dataset",
    expected_output=""
    JSON con estructura:
    {
        "summary": "Resumen de 1 párrafo",
        "key_findings": ["finding1", "finding2", ...],
        "recommendations": ["rec1", "rec2", ...],
        "confidence_score": 0-100
    }
    ""
)

```

9.2. 9.2 Debugging Avanzado

Técnicas para diagnosticar problemas:

```

# 1. Aumentar verbosidad
crew = Crew(
    agents=agents,
    tasks=tasks,
    verbose=True,
    log_file="debug.log"
)

# 2. Ejecutar tasks independientemente
result = task1.execute()
print(f"Task 1 output: {result}")

# 3. Verificar agent memory
print(agent.memory.get_memory())

# 4. Probar herramientas aisladamente
result = search_tool.execute("test query")
print(result)

# 5. Usar try-except
try:
    result = crew.kickoff()
except Exception as e:
    print(f"Error: {e}")
    import traceback
    traceback.print_exc()

```

Chapter 10. Módulo 10: Proyecto Final

10.1. 10.1 Proyecto: Sistema de Análisis de Tendencias Multi-Fuente

Objetivo: Crear un sistema completo que investigues, analices y predijas tendencias de mercado usando múltiples agentes coordinados.

10.1.1. Requisitos del Sistema

1. INVESTIGACIÓN

- - Buscar tendencias en múltiples fuentes
 - Validar información
 - Clasificar por relevancia

2. ANÁLISIS

- - Extraer insights
 - Identificar patrones
 - Calcular impacto potencial

3. PREDICCIÓN

- - Proyectar tendencias futuras
 - Estimar timelines
 - Calcular probabilidades

4. REPORTE

- - Sintetizar hallazgos
 - Crear visualizaciones
 - Exportar en múltiples formatos

10.1.2. Arquitectura del Proyecto

```
from crewai import Agent, Task, Crew, Process

# AGENTES ESPECIALIZADOS
researcher = Agent(
    role="Market Researcher",
    goal="Identificar tendencias emergentes",
    backstory="10+ años en research de mercado",
    tools=[search_tool, web_scraper]
)
```

```

analyst = Agent(
    role="Data Analyst",
    goal="Extraer insights de datos",
    backstory="PhD en Estadística",
    tools=[data_analyzer, visualization_tool]
)

forecaster = Agent(
    role="Trend Forecaster",
    goal="Predecir tendencias futuras",
    backstory="Especialista en forecasting",
    tools=[statistical_tool]
)

reporter = Agent(
    role="Report Writer",
    goal="Presentar hallazgos profesionalmente",
    backstory="Writer con 15 años de experiencia"
)

# TAREAS COORDINADAS
research_task = Task(
    description="Investiga tendencias en {market}",
    expected_output="Reporte con 10 tendencias",
    agent=researcher
)

analysis_task = Task(
    description="Analiza las tendencias investigadas",
    expected_output="Análisis con insights",
    agent=analyst,
    depends_on=[research_task]
)

forecast_task = Task(
    description="Predice evolución de tendencias",
    expected_output="Predicciones con probabilidades",
    agent=forecaster,
    depends_on=[analysis_task]
)

report_task = Task(
    description="Genera reporte ejecutivo final",
    expected_output="Reporte en PDF",
    agent=reporter,
    depends_on=[forecast_task]
)

# CREW PRINCIPAL
crew = Crew(
    agents=[researcher, analyst, forecaster, reporter],

```

```
tasks=[research_task, analysis_task, forecast_task, report_task],
process=Process.sequential,
verbose=True
)

# EJECUCIÓN
result = crew.kickoff(inputs={"market": "Inteligencia Artificial"})
print(result.raw_output)
```

10.1.3. Entregables Esperados

1. **Código fuente** con estructura profesional
2. **Documentación técnica** del sistema
3. **Reporte de tendencias** generado por el sistema
4. **Predicciones** con análisis de confianza
5. **Guía de uso** y configuración

10.2. 10.2 Presentación y Validación

Criterios de éxito:

- ✓ Sistema ejecuta sin errores
- ✓ Todos los agentes se ejecutan correctamente
- ✓ Outputs tienen calidad y están en formato esperado
- ✓ Sistema es escalable a nuevos agentes/tareas
- ✓ Documentación es clara y completa
- ✓ Código es mantenible y reutilizable

Chapter 11. Anexo A: Comparativa de Frameworks

CrewAI vs AutoGen vs LangChain - Matriz de decisión por caso de uso.

Chapter 12. Anexo B: Recursos

- Documentación oficial
- Ejemplos de la comunidad
- Troubleshooting guide

Chapter 13. Anexo C: Glosario

Términos clave y conceptos de agentes.

Chapter 14. Anexos

14.1. A. Comparativa de Frameworks

. * CrewAI vs AutoGen vs LangChain * Matriz de decisión * Selección por caso de uso

14.2. B. Recursos

. * Documentación oficial * GitHub repositories * Ejemplos comunitarios * Discord community

14.3. C. Glosario

. * Términos clave * Acrónimos * Conceptos de agentes * Referencias

14.4. D. Checklist de Proyecto

. * Planificación * Desarrollo * Testing * Optimización * Deployment